

Report for the Course on Cyber-Physical Systems and IoT Security

Replay Attack Detection in a Platoon of Connected Vehicles with Cooperative Adaptive Cruise Control

Boscolo Meneguolo Luca - 2113488

1. Objective

Cooperative ACC is the direct follow-up of the Adaptive Cruise Control, a device that allows to keep a desired headway to the preceding vehicle. This technology can be used to setup platooning, a sequence of vehicles where one follow each other adaptively at safe distance. This will in turn reduce traffic, as the platoon is managed by fast-responsive controllers, and possibly reduce the pollution by increasing the aerodynamic efficiency. However, as we delegate human responsibility to machines, attacks can be attempted to harm the stability of the system, by degrading the performance or causing severe accidents. In this work, we will refer to the paper [1] to demonstrate the usage of a basic platooning simulator to determine how replay attacks are implemented, and a possible solution that can detect them. Moreover, an extension on the authors' work was conducted to explore the possibility for a lightweight implementation of the detection algorithm and possible attacks to the countermeasure.

The lineup is the following:

- Build a basic platooning simulator,
- Perform a replay attack,
- Attack detection,
- Analysis on the feasibility of the optimization strategy,
- Attacks from the leader.

2. System Setup

This project requires to build a basic simulator from scratch. The preferred choice was to use Python as programming language, as it's very convenient. Moreover, Python offers a wide

range of libraries that are very practical. In particular NumPy library was used to perform matrices operations, and Matplotlib library to draw plots. Colab was chosen as Google's cloud implementation of Jupyter Notebooks as is very convenient to perform iterated tests.

3. Experiments

The core challenge of this project is to demonstrate a Cooperative Adaptive Cruise Control (CACC) simulator in action. CACC allows the involved vehicles to follow the preceding vehicle by keeping a desired relative distance by exploiting the wireless communication and the onboard sensors, i.e. radar. In a CACC platoon, every vehicle has access to the preceding vehicle's speed and acceleration, and manages to measure the inter vehicle distance through onboard independent sensors. The attached Jupyter Notebook contains all the details of the Python code.

3.1. Simulator

The simulator serves the purpose to conduct tests on a platoon of k vehicles, and thus was designed to permit the full customizability of all parameters, to ensure easy tuning. The core of the simulator consists on the representation of the system.

The vehicles are numbered in ascending order, starting from the leader as vehicle 0 and following up to vehicle $k - 1$.

The state of each vehicle is represented by a vector

$$x_i(t) = \begin{bmatrix} d_i(t) \\ v_i(t) \\ a_i(t) \end{bmatrix}$$

where $d_i(t)$ is the distance between vehicle i and $i - 1$, and $v_i(t)$ and $a_i(t)$ are respectively the velocity and the acceleration of vehicle i . The component $d_0(t)$ is obviously not defined for the system.

We assume that all vehicles can only move along one axis, i.e. only forward and backwards. It's also assumed that the protocol allows for the vehicles to share informations about their velocity and acceleration to the following ones. Such communication happens instantaneously, and at the maximum frequency possible, i.e. it updates every time the system updates.

The leader is the only vehicle that is controllable by a predefined acceleration profile u_r . The profile is constructed as a table, where for each point in time there is a corresponding acceleration value.

The leader vehicle then determines the jerk $j_0(t) = \frac{du_r(t)}{dt}$, and uses the following calculation to update its state:

$$x_0(t+1) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} \cdot x_0(t) + \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \cdot j_0(t)$$

Subsequent vehicles, instead, make use of the radar measurement and the feedback from the preceding vehicle to calculate their error:

$$e_i(t) = q_{i-1}(t) - q_i(t) - h \cdot v_i(t)$$

where $q_i(t)$ is the absolute distance along the axis.

$e_i(t)$ represents the error between the desired headway h and the one measured by the radar, i.e. how much the vehicle is off compared to the desired distance. The error is computed to scale the input jerk of car i accordingly.

$$j_i(t+1) = \frac{1}{h} [-a_i(t) + k_p e_i(t) + k_d \dot{e}_i(t) + a_{i-1}(t)]$$

Once obtained the jerk, use the same formula to calculate and apply the new state.

These calculations can be simplified in a one line matrix expression:

$$x_i(t+1) = A \cdot x_i(t) + b \cdot v_{i-1}(t) + c \cdot a_{i-1}(t)$$

where

$$A = \begin{bmatrix} 1 & -1 & 0 \\ 0 & 1 & 1 \\ \frac{k_p}{h} & -(k_p + \frac{k_d}{h}) & 1 - (k_d + \frac{1}{h}) \end{bmatrix},$$

$$b = \begin{bmatrix} 1 \\ 0 \\ \frac{k_d}{h} \end{bmatrix}, \quad c = \begin{bmatrix} 0 \\ 0 \\ \frac{1}{h} \end{bmatrix},$$

h is the desired headway distance between vehicles in ms, and parameters k_p and k_d are used to tune the response of the following vehicles.

Each simulation must be initialized with these parameters:

- `MAX_TIME`: the maximum time of the simulation in milliseconds;
- `T`: the resolution of the time unit in seconds;
- `k`: the number of vehicles that compose the platoon;
- `h`: the target heading between vehicles in milliseconds.

Snippet 1 shows the implementation of the basic simulator.

3.2. Replay Attack

A very powerful attack in a CACC platoon is the Replay Attack. It's assumed that the attacker has full control over the CACC control logic, and can override the controls. More specifically, the attacker can spoof fake values for the transmitted velocities and accelerations. A more demanding assumption is that the leader is supposed to be always legitimate. This comes from a simplification that the authors of the paper propose to facilitate their analysis, but it may not hold in the real world. An experiment included on this report will indeed show this vulnerability in action.

When the platoon is in a steady state, we can note that all velocities are constant, and all accel-

erations are 0 m/s^2 . This scenario is typical in highways. In this phase, the attacker will record real time velocities and accelerations from its vehicle and store them in a buffer. When the malicious vehicle initiates the attack, it starts to replay the recorded values to the the following vehicles. This in general has the first effect of degrading the performance of the platoon, as small variations on the speed are not compensated by the headway error calculation. However, it becomes very dangerous in a situation when the leader decides to decelerate: the vehicle behind the attacker will unavoidably crash onto it, because it was receiving spoofed steady-state values. Chain crashes are highly probable as well.

Snippet 2 implements a basic replay attack. The base code is very similar to the normal simulator,

it only differs for the logic to record and replay the values.

3.3. Countermeasure – dual model detection

A possible countermeasure proposed by the authors requires very simple modifications.

The leader needs to be equipped with a pseudo-random number generator, that samples from a gaussian distribution with mean 0 and variance in the order of 10^{-5} . This assumes again that the leader is always legitimate and generates random numbers according the distribution.

These random generated noises will be added to the input acceleration given by the profile:

$$\Delta u_r \sim \mathcal{N}(0, 10^{-5})$$

$$\tilde{u}_r = u_r + \Delta u_r$$

```
prev_acc = 0
for time in range(MAX_TIME):
    if(time < 15000): ur = 4          # definition of the acceleration profile
    else: ur = 0

    for i in range(k):
        if(i == 0):                  # first vehicle
            jerk = (ur - prev_acc)*T*T
            prev_acc = ur
            x[i] = np.dot(A0, x[i]) + np.dot(b0, jerk)
        else:                         # all other vehicles
            x[i] = np.dot(Ai, x[i]) + np.dot(bi, x[i-1][1]) + np.dot(ci, x[i-1][2])
```

Snippet 1: Basic simulator.

```
for i in range(k):
    if(i == 0):
        jerk = (ur - prev_acc)*T*T
        prev_acc = ur
        x[i] = np.dot(A0, x[i]) + np.dot(b0, jerk)
    elif i == attacker:
        if time >= START_RECORDING and time < START_RECORDING + RECORD_SIZE:
            ei = x[i][0] - h*x[i][1]
            dei = x[i-1][1] - x[i][1] - h*x[i][2]
            jerk = (x[i-1][2] - x[i][2] + (kp*ei + kd*dei)) / h
            recorded_jerks[time - START_RECORDING] = jerk
            x[i] = np.dot(Ai, x[i]) + np.dot(bi, x[i-1][1]) + np.dot(ci, x[i-1][2])
        elif i != attacker+1:
            x[i] = np.dot(Ai, x[i]) + np.dot(bi, x[i-1][1]) + np.dot(ci, x[i-1][2])
        elif i == attacker+1:
            if time >= attack_start:
                jerk = recorded_jerks[(time - attack_start) % RECORD_SIZE]
                dxi = np.dot(A, x[i]) + np.dot(b, x[i-1][1]) + np.dot(c, jerk)
                x[i] = x[i] + dxi
            else:
                x[i] = np.dot(Ai, x[i]) + np.dot(bi, x[i-1][1]) + np.dot(ci, x[i-1][2])
```

Snippet 2: Replay attack.

The core is to make the deltas as small as possible to make it impossible for the users of the platoon to feel the difference, to avoid sickness. The leader, other than broadcasting the velocity and acceleration to the system, will also broadcast those random generated values.

These small perturbations on the acceleration will have a direct effect of changing the error $e_i(t)$, and thus propagate to the following vehicles as variations of their acceleration.

Every vehicle's control unit has a virtual model running. The virtual model's purpose is to estimate the true expected values for the acceleration of the preceding car in absence of attacks, based on the shared random noise. It's possible to cross correlate the acceleration of the virtual model, and the acceleration that vehicle i calculates on the real model. If the cross correlation is approximately 0, with high probability the acceleration that vehicle i calculated are not timely according to the leader. Thus with high probability an attack is occurring. Upon discovery, the vehicle might interrupt the normal navigation and perform safe operations, like increase the headway distance or slowly come to a stop.

To test this countermeasure, the random number generator needs to be added on the leader vehicle:

```
random = np.random.normal(0, np.sqrt(0.00001))
ur = ur + random
```

The cross correlation can be computed in windows of different size. It might be logic to assume that the bigger the window, the more precise the result will be, at the expense of detection speed. Windows sizes of 15, 10, and 5s were tested.

3.4. Countermeasure – detect the attack using only the noise

The virtual model suggested by the authors requires additional computational power, and can be a problem for long platoons, as every vehicle must simulate the dynamics of all the preceding vehicles.

A more lightweight implementation could be implemented by cross correlating the calculated acceleration directly with the generated noise, instead of simulating the acceleration using the

virtual model. This could lead to promising results given the fact that the cross correlation is a powerful function that computes the level of similarity between two signals as a function of a time lag applied to one of them.

Since the noise directly affects the accelerations, we can expect these similarities to be reflected by both of the signals. This implementation is very lightweight and doesn't require each car to recursively simulate the preceding vehicles.

3.5. Countermeasure of the countermeasure – leader is adversary

The authors of the paper correctly state that the results hold within the assumption that the leader is immune to attacks. This is in general not true and a series of attacks that affect the leader can pose severe issues to this protocol.

Some attacks will be discussed and tested on the result section.

4. Results and Discussion

4.1. Simulator

The plot in Figure 1 shows the result of the simulation using the following parameters:

$MAX_TIME = 80000$ ms, $T = \frac{1}{1000}s$, $k = 5$,

$h = 600$ ms.

The leader follows this acceleration profile:

$$u_r = \begin{cases} 4 & \text{if time} < 5000 \\ 0 & \text{if time} < 15000 \\ 1 & \text{if time} < 30000 \\ 0 & \text{if time} < 68000 \\ -5 & \text{if time} < 75000 \\ 0 & \text{else} \end{cases}$$

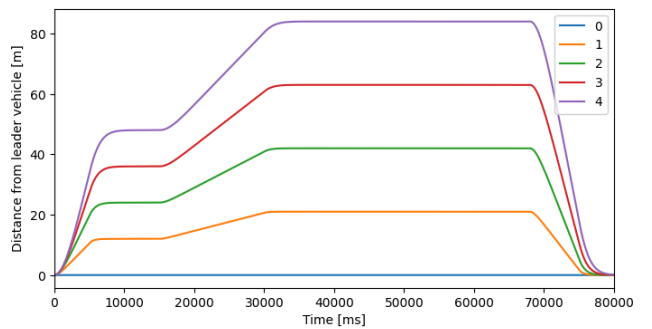


Figure 1: Basic simulation.

From the plot in Figure 1, we can see that the vehicles behave as expected, keeping the desired headway in all phases of the simulation. The distance in meters between the vehicles depends on the speed, and is well uniformed accross all of them. At the end of the simulation, we can see that the leader stops and all other vehicles follow. This offers the baseline to perform all the future experiments.

4.2. Replay Attack

The plot in Figure 2 shows the result of the simulation using the following parameters:

$\text{MAX_TIME} = 80000 \text{ ms}$, $\tau = \frac{1}{1000} \text{ s}$, $k = 4$,

$h = 600 \text{ ms}$, $\text{attacker} = 2$,

$\text{attack_start} = 40000 \text{ ms}$, $\text{RECORD_SIZE} = 5000 \text{ ms}$,

$\text{RECORD_START} = 20000 \text{ ms}$.

The leader follows this acceleration profile:

$$u_r = \begin{cases} 4 & \text{if time} < 5000 \\ 0 & \text{if time} < 60000 \\ -4 & \text{if time} < 65000 \\ 0 & \text{else} \end{cases}$$

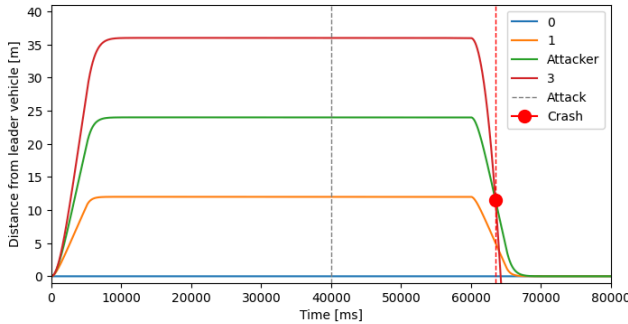


Figure 2: Replay attack simulation. Crash occurs at time 63495 ms.

It's possible to note that, on the first phase, all four cars behave, and accelerate according to the specifications. From time $> 20000 \text{ ms}$, and for a duration of 5000 ms, the attacker records its real world values. Then, it replays those values ciclically from time $> 40000 \text{ ms}$. At time $= 60000 \text{ ms}$, the leader starts to decelerate, and the following vehicles, including the attacker, follow accordingly. The vehicle behind the attacker, though, received spoofed values and quickly crashes onto it.

4.3. Countermeasure – dual model detection

In Figure 3 we can see the outcome of the simulation. The simulation is a long run of 1000 s , where 2 distinct replay attacks take place. The first one starts at 300 s and ends at 450 s , and the second one starts at 700 s and ends at 800 s . During the time the attack is not happening, the cross correlation is well over 0. Moreover, the previous assumption is correct, as it's possible to find a stronger correlation with larger windows. During each of the two attacks, the cross correlation quickly drops to 0 in all three cases.

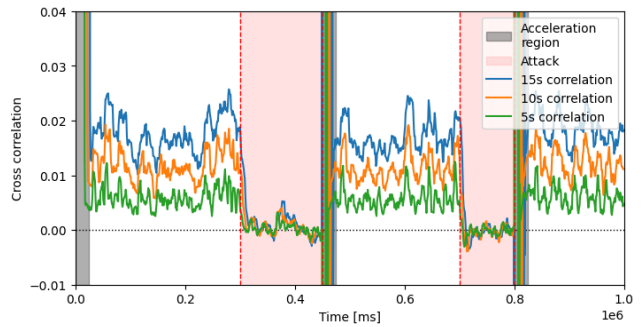


Figure 3: Cross correlation calculated using the virtual model.

The areas highlighted in grey are marked as acceleration regions. We can see that inside this area, the values for cross correlation are way off the expected. Coincidentally, those moments happen concurrently to large accelerations from the vehicle.

When the plateau is in the initial acceleration phase, every small perturbation is really not going to influence by much the value of the error $e_i(t)$. The vehicle has to pick up the speed and the noise is just too irrelevant to have a noticeable impact. After each attack, the vehicle starts receiving true values, and has to correct for a possible error that accumulated during the attack. Because of this, an acceleration is induced, making the value of the correlation inconsistent again. The phases when the acceleration is 0, instead, are when the small perturbations truly influence the next vehicles' acceleration, making the detection possible.

We can surely assume that this countermeasure demonstrated powerful at detecting replay attacks during steady-state platoons. From these

results, it seems like under strong accelerations we don't have a usable correlation. Future work could aim at increasing the noise variance, so that more perturbation is induced, at the cost of possible sickness of the users.

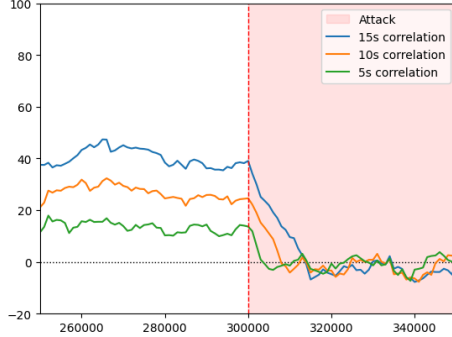


Figure 4: Closeup of the start of an attack.

In Figure 4 we see a close up on the time where the first attack takes place. Specifically, we can see in more detail the time each correlation takes to respond, under all the three window sizes. The 5-seconds correlation, despite being weaker, reacts very quickly. The 15-seconds correlation is certainly stronger, but requires extra seconds to fully nullify at 0. A tradeoff between accuracy and timeliness is offered by the 10s correlation.

4.4. Countermeasure – detect the attack using only the noise

In Figure 5 we can see the outcome of the simulation. The simulation parameters are analogous as the previous experiment, the only difference is how the vehicle i computes the cross correlation.

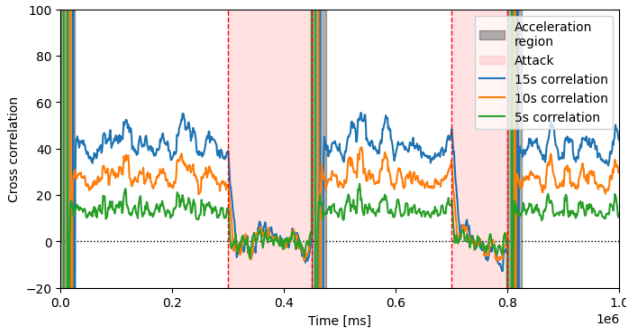


Figure 5: Cross correlation between the random generated values and the acceleration computed by the vehicle after the attacker.

We can clearly see that the results are promising: cross correlating directly the random noise with the values received by the preceding vehicle can

detect the attack. This reduces the computations needed to perform the detection in large-sized platoons.

However, this approach will work only if the platoon has a number of vehicles such that each perturbation from the leader affects the vehicle i within the correlation window.

This is not guaranteed, as we can possibly have more than 50 vehicles on a platoon that would require a window size of more than 1 minute. We also showed before that the longer the window size, the longer the detection time.

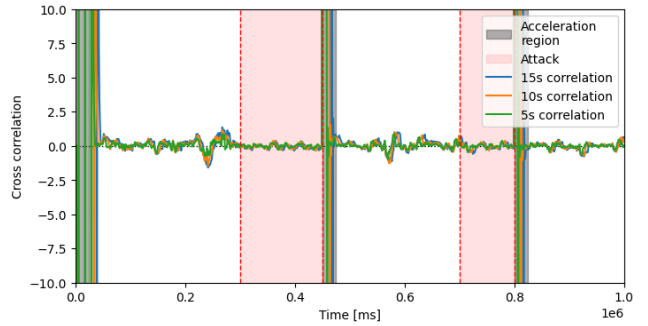


Figure 6: Simplified cross correlation for a platoon of 20 cars: there are too many vehicles compared to the correlation windows, and the correlation is always 0

In conclusion, although the solution proposed by the authors is more computationally demanding for long platoons, it's the only option that grants security. But also for short platoons it is not useful, as we don't have a high computational overhead. Running the full virtual model is thus the only choice.

4.5. The leader as adversary

There are attacks involving the leader that fail to grant the security of the platoon.

4.5.1. All generated values are constant

This attack assumes that the leader has access to the random number generator, which is a strong assumption.

$$\Delta u_r = c$$

With this attack, a constant is added to the input acceleration. In this case, there isn't any randomness, so any replay attack wouldn't be noticed.

However, this attack is too trivial and we also have an assumption issue: we cannot always assume that the leader has access to the control logic, as there might be precautions into that. Another attack must be designed.

4.5.2. Input manipulation

One parameter where the leader has full control is the acceleration profile, and this is according to all the assumptions we made before. We can design an acceleration profile as follows.

Once steady state condition is reached:

$$u_r = \begin{cases} 0.5 & \text{if time is even} \\ -0.5 & \text{if time is odd} \end{cases}$$

meaning that every millisecond, the control will apply an acceleration whose module is dependant on the parity. These accelerations are not felt by the users of the platoon, as they only last for one millisecond.

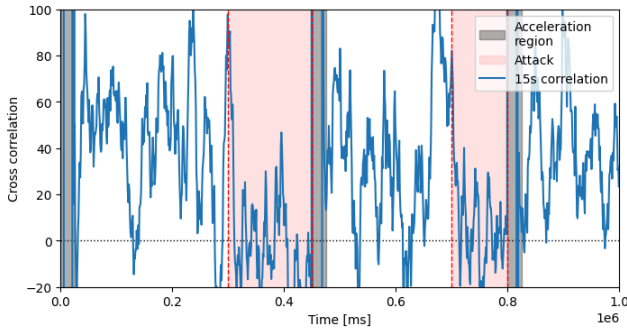


Figure 7: Cross correlation under the attack.

In Figure 7 we can see a simulation of such attack. The simplified algorithm was used, as the platoon is short and its performance was showed. The plot shows a very noisy cross correlation: it's very difficult to establish a threshold that determines the attack. What could be beneficial is to compute the average across a few seconds, and in turn should filter out most of the noise. However, computing the average adds a delay on the detection, and the results might still not be usable to detect an attack. Also, a higher modulo could be added by the leader, resulting in more noise.

This attack is very powerful because it only leverages the input acceleration, which is totally up to the leader by assumption.

5. Conclusions

In this work we showed that CACC is an interesting technology that allows to setup platoons. They can drastically improve the responsiveness of traffic and thus can be fundamental in big crowded cities. However, there are some attacks like the replay attack that can threaten the safety of users. A very interesting algorithm developed by the authors of the paper can detect such attack, but future work is needed to find a solution of the leader's immunity assumption.

Bibliography

- [1] R. Merco, A. B. Zoleikha, and P. Pisu, "Replay Attack Detection in a Platoon of Connected Vehicles with Cooperative Adaptive Cruise Control." [Online]. Available: <https://ieeexplore.ieee.org/document/8431538>